

PRACTICAL WEEK-5

A SANJANA REDDY

2520090159

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Navigator' pane shows the 'MANAGEMENT' section with options like Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, and Data Import/Restore. Below this is the 'INSTANCE' section with Startup / Shutdown, Server Logs, and Options File. The 'PERFORMANCE' section includes Administration and Schemas. The 'Information' section at the bottom shows 'No object selected'.

The main window, titled 'Query 1 library_db', shows a query execution result. The query is 'CREATE DATABASE library_db;'. The output is displayed in a table with columns: #, Time, Action, and Message.

#	Time	Action	Message
7	20:05:53	SELECT * FROM students LIMIT 0, 50000	3 row(s) returned
8	09:38:32	CREATE DATABASE library_db	1 row(s) affected

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Administration
- Schemas

Information

No object selected

Query 1 library_db

```

1 CREATE DATABASE library_db;
2 CREATE TABLE IF NOT EXISTS members (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     username VARCHAR(100) UNIQUE NOT NULL,
5     password_hash VARCHAR(255) NOT NULL,
6     role VARCHAR(30) NOT NULL DEFAULT 'member'
7 );
8
9 CREATE TABLE IF NOT EXISTS books (
10    id INT AUTO_INCREMENT PRIMARY KEY,
11    title VARCHAR(250) NOT NULL,
12    author VARCHAR(200) NOT NULL,
13    isbn VARCHAR(13) UNIQUE NOT NULL,
14    price DECIMAL(10,2) NOT NULL,
15    published_date DATE NOT NULL,
16    description TEXT,
17    embedding JSON
18 );
19
20
21

```

Output

Action Output

#	Time	Action	Message
8	09:38:32	CREATE DATABASE library_db	1 row(s) affected
9	09:38:53	CREATE DATABASE library_db	Error Code: 1007. Can't create database 'library_db'; database exists
10	09:40:58	CREATE TABLE IF NOT EXISTS members (id INT AUTO_INCREMENT PRIMARY KEY, username VAR...	0 row(s) affected

Object Info Session

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Administration
- Schemas

Information

No object selected

Query 1 library_db

```

1 CREATE DATABASE library_db;
2 CREATE TABLE IF NOT EXISTS members (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     username VARCHAR(100) UNIQUE NOT NULL,
5     password_hash VARCHAR(255) NOT NULL,
6     role VARCHAR(30) NOT NULL DEFAULT 'member'
7 );
8
9 CREATE TABLE IF NOT EXISTS books (
10    id INT AUTO_INCREMENT PRIMARY KEY,
11    title VARCHAR(250) NOT NULL,
12    author VARCHAR(200) NOT NULL,
13    isbn VARCHAR(13) UNIQUE NOT NULL,
14    price DECIMAL(10,2) NOT NULL,
15    published_date DATE NOT NULL,
16    description TEXT,
17    embedding JSON
18 );
19
20
21

```

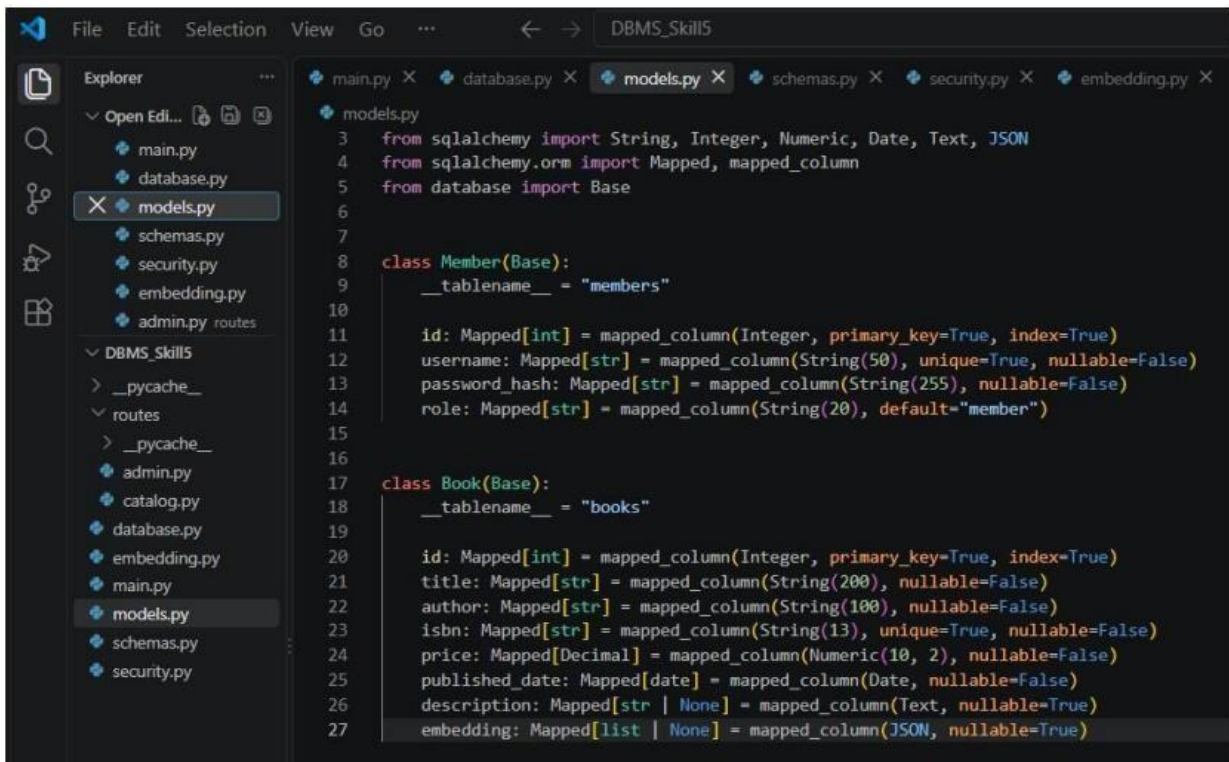
Output

Action Output

#	Time	Action	Message
9	09:38:53	CREATE DATABASE library_db	Error Code: 1007. Can't create database 'library_db'; database exists
10	09:40:58	CREATE TABLE IF NOT EXISTS members (id INT AUTO_INCREMENT PRIMARY KEY, username VAR...	0 row(s) affected
11	09:41:27	CREATE TABLE IF NOT EXISTS books (id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(2...	0 row(s) affected

Object Info Session

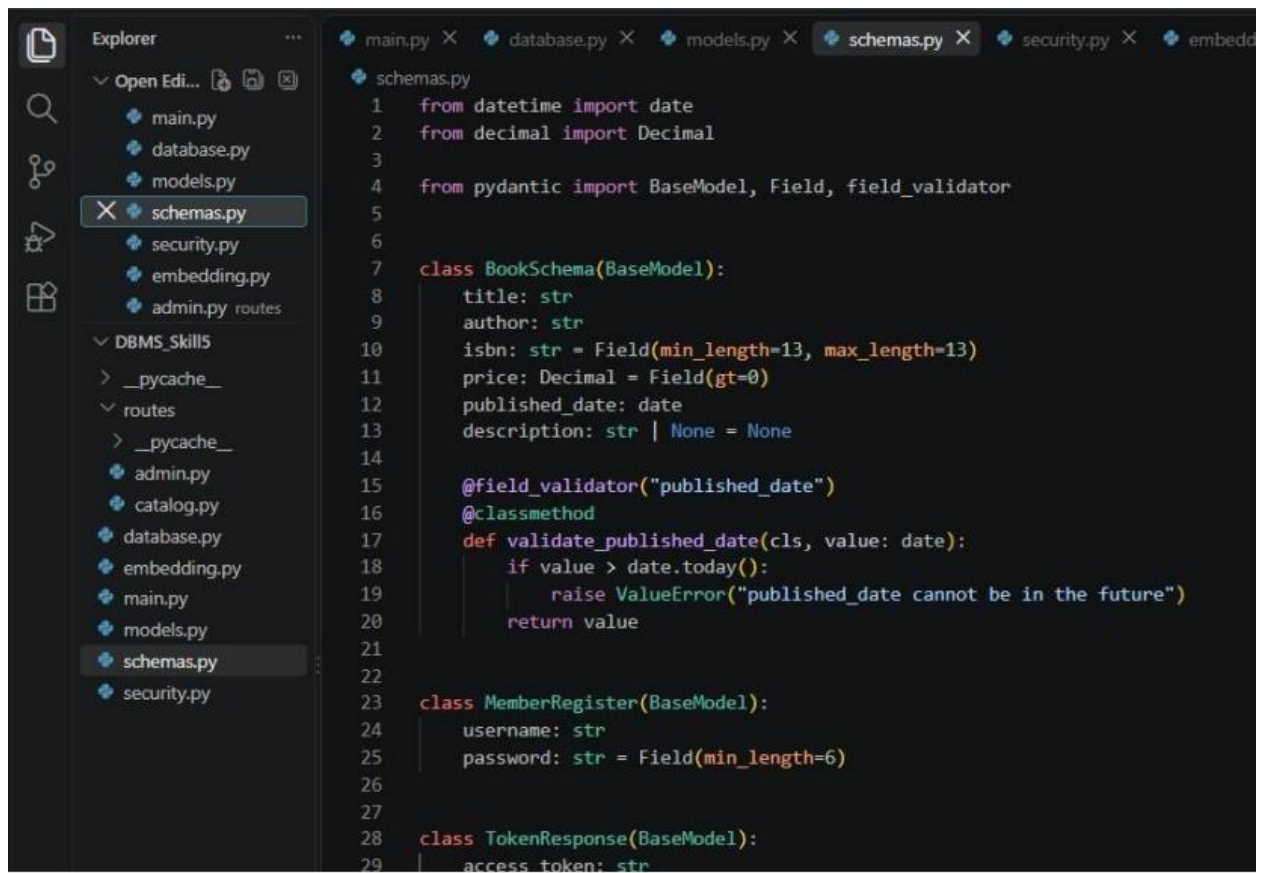
```
PS C:\library_api> python -m pip install fastapi uvicorn
>> python -m pip install "python-jose[cryptography]"
>> python -m pip install passlib[bcrypt]
● >> python -m pip install python-multipart
```

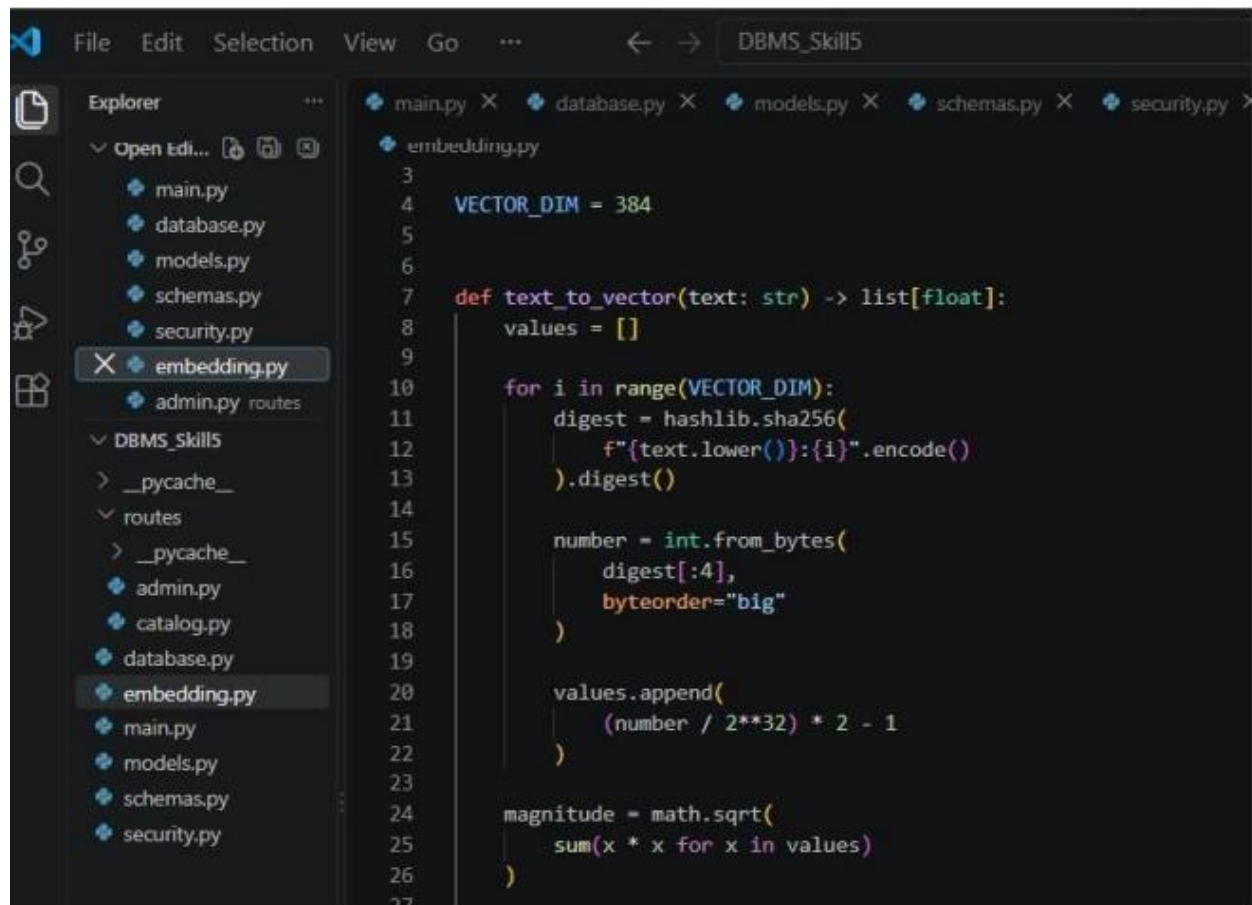


```
File Edit Selection View Go ...
DBMS_Skills

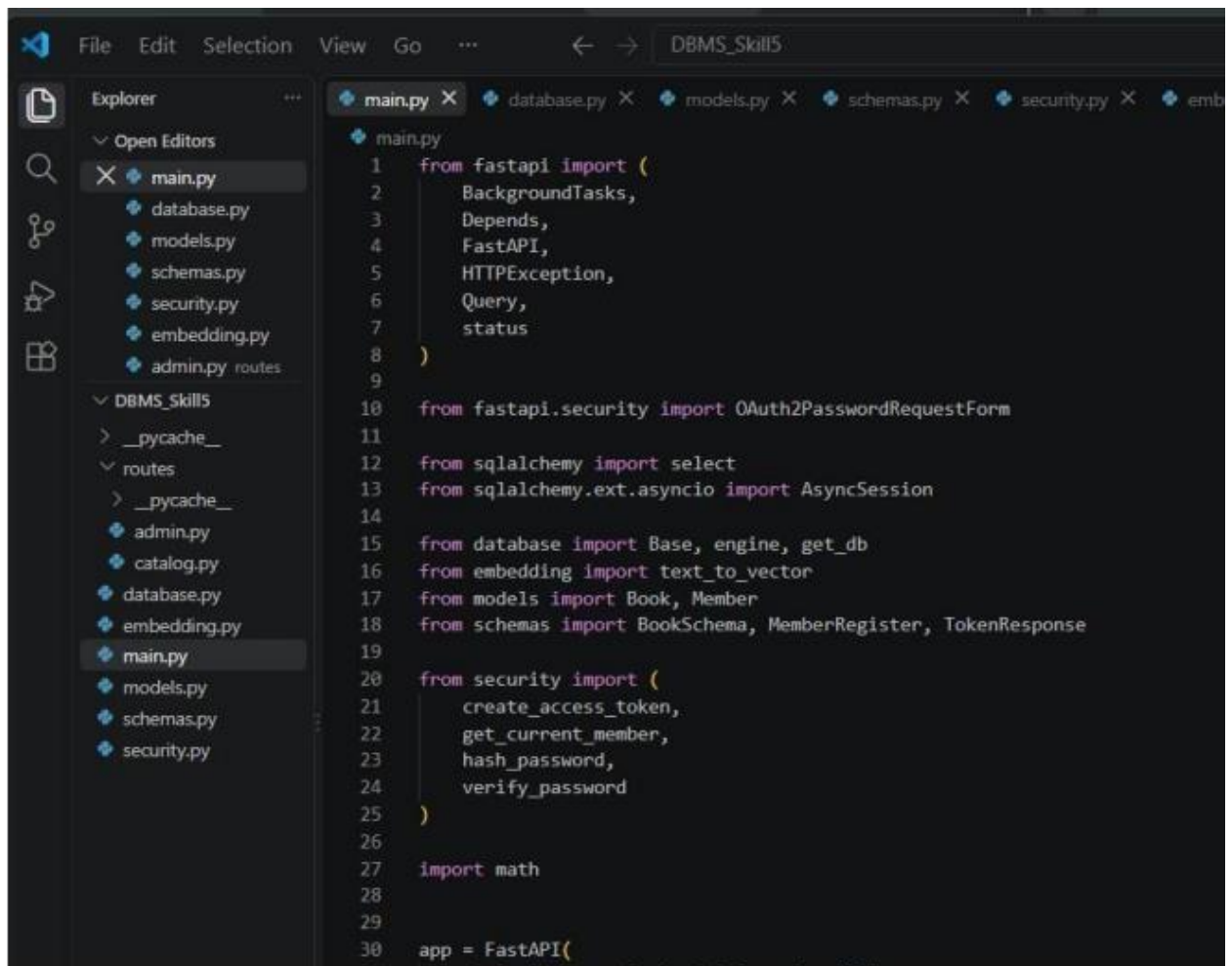
Explorer
  Open Ed...
    main.py
    database.py
    X models.py
    schemas.py
    security.py
    embedding.py
    admin.py routes
  DBMS_Skills
    > __pycache__
    > routes
    > __pycache__
    admin.py
    catalog.py
    database.py
    embedding.py
    main.py
    models.py
    schemas.py
    security.py

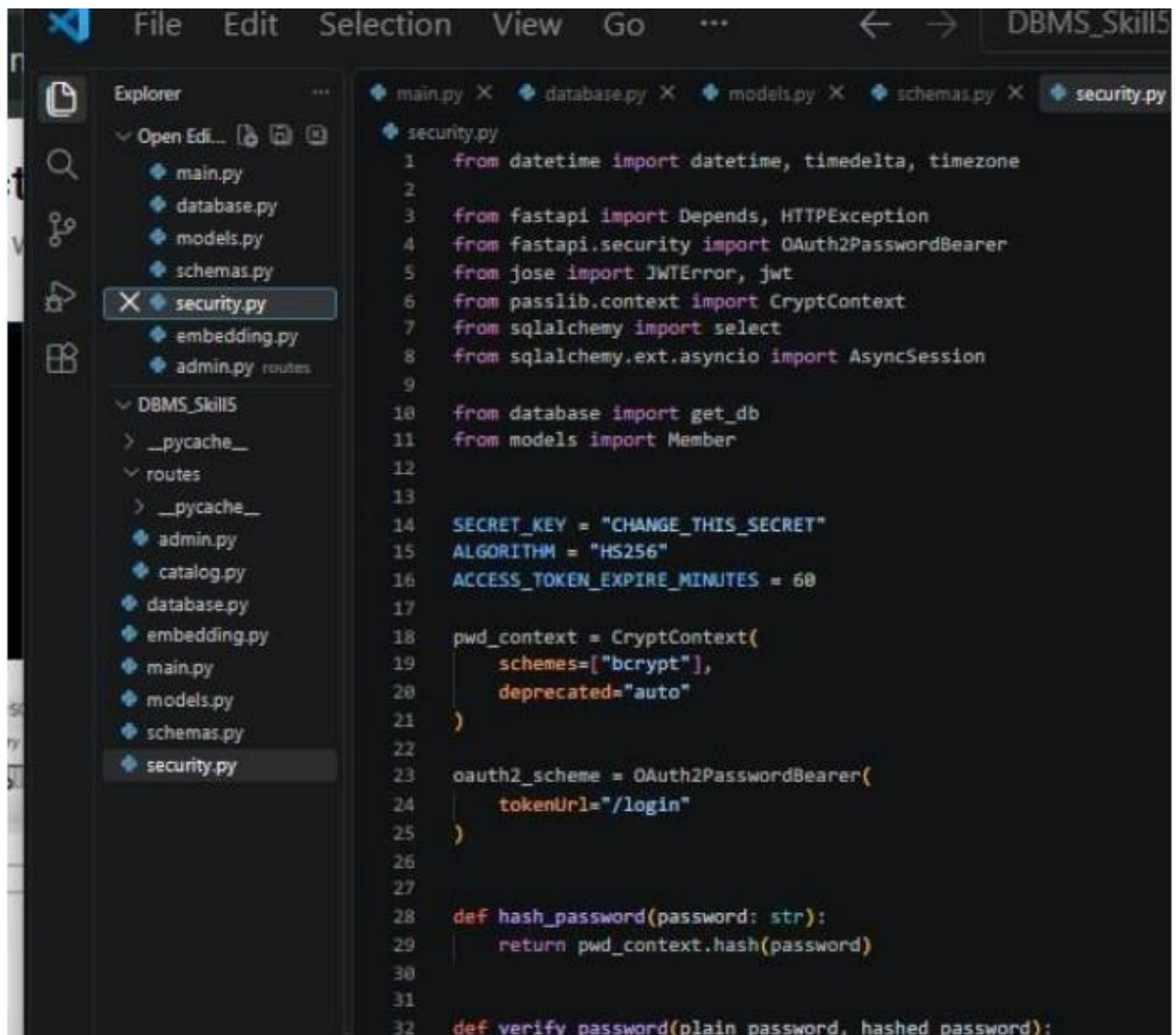
models.py
3 from sqlalchemy import String, Integer, Numeric, Date, Text, JSON
4 from sqlalchemy.orm import Mapped, mapped_column
5 from database import Base
6
7
8 class Member(Base):
9     __tablename__ = "members"
10
11     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
12     username: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
13     password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
14     role: Mapped[str] = mapped_column(String(20), default="member")
15
16
17 class Book(Base):
18     __tablename__ = "books"
19
20     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
21     title: Mapped[str] = mapped_column(String(200), nullable=False)
22     author: Mapped[str] = mapped_column(String(100), nullable=False)
23     isbn: Mapped[str] = mapped_column(String(13), unique=True, nullable=False)
24     price: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
25     published_date: Mapped[date] = mapped_column(Date, nullable=False)
26     description: Mapped[str | None] = mapped_column(Text, nullable=True)
27     embedding: Mapped[list | None] = mapped_column(JSON, nullable=True)
```





```
3
4 VECTOR_DIM = 384
5
6
7 def text_to_vector(text: str) -> list[float]:
8     values = []
9
10    for i in range(VECTOR_DIM):
11        digest = hashlib.sha256(
12            f"{text.lower():{i}".encode()
13        ).digest()
14
15        number = int.from_bytes(
16            digest[:4],
17            byteorder="big"
18        )
19
20        values.append(
21            (number / 2**32) * 2 - 1
22        )
23
24    magnitude = math.sqrt(
25        sum(x * x for x in values)
26    )
27
```





The image shows a code editor interface with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, and navigation arrows. The Explorer sidebar on the left shows a project structure with folders like DBMS_Skills and files like security.py. The main editor window displays the content of security.py, which includes imports for datetime, timedelta, timezone, fastapi, jose, passlib, sqlalchemy, and database. It also defines constants for SECRET_KEY, ALGORITHM, and ACCESS_TOKEN_EXPIRE_MINUTES, and functions for hashing and verifying passwords.

```
File Edit Selection View Go ... DBMS_Skills5
main.py X database.py X models.py X schemas.py X security.py
security.py
1 from datetime import datetime, timedelta, timezone
2
3 from fastapi import Depends, HTTPException
4 from fastapi.security import OAuth2PasswordBearer
5 from jose import JWTError, jwt
6 from passlib.context import CryptContext
7 from sqlalchemy import select
8 from sqlalchemy.ext.asyncio import AsyncSession
9
10 from database import get_db
11 from models import Member
12
13
14 SECRET_KEY = "CHANGE_THIS_SECRET"
15 ALGORITHM = "HS256"
16 ACCESS_TOKEN_EXPIRE_MINUTES = 60
17
18 pwd_context = CryptContext(
19     schemes=["bcrypt"],
20     deprecated="auto"
21 )
22
23 oauth2_scheme = OAuth2PasswordBearer(
24     tokenUrl="/login"
25 )
26
27
28 def hash_password(password: str):
29     return pwd_context.hash(password)
30
31
32 def verify_password(plain_password, hashed_password):
```

```
File Edit Selection View Go ... DBMS_Skills5
main.py X database.py X models.py X schemas.py X security.py X embedding.py X
security.py
39 def create_access_token(username, role):
50     return jwt.encode(
51         SECRET_KEY,
52         algorithm=ALGORITHM
53     )
54
55
56
57 async def get_current_member(
58     token: str = Depends(oauth2_scheme),
59     db: AsyncSession = Depends(get_db)
60 ):
61     credentials_exception = HTTPException(
62         status_code=401,
63         detail="Invalid or expired token",
64         headers={"WWW-Authenticate": "Bearer"}
65     )
66
67     try:
68         payload = jwt.decode(
69             token,
70             SECRET_KEY,
71             algorithms=[ALGORITHM]
72         )
73
74         username = payload.get("sub")
75
76         if not username:
77             raise credentials_exception
78
79     except JWTError:
80         raise credentials_exception
81
82     result = await db.execute(
83         select(Member).where(
84             Member.username == username
85         )
86     )
87
88     member = result.scalar_one_or_none()
89
90     if not member:
91         raise credentials_exception
92
93     return member
```